MLE basics — learn the draft fit from scratch

For: Charles — instructional (not a quick-reference card)
Last synced: 2026-08-20
Start here if you want the story first: `MLE_fit_explainer.md`
Older pipeline context: ASSIGN / SCORE / SELECT, ρ*, hero POST-QC — assumed; **MLE concepts are not.**

Part 0 — Committed fit (where this lands)

After running `pd21_draft_bernoulli_mle.py` on 2013–2021 POST-QC panel (min 20 min):

Quantity	Value
Player-seasons	38,123
Drafted (Y = 1)	882
λ̂	≈ 2.57
t̂	≈ 1.07
γ	18 (fixed for committed run)
Best log-likelihood ℓ	≈ −6919.1

Artifacts: `../re_entry/HEROs_and_PASSes/pd21_mle/`
The rest of this document explains **how you get there**, not just the numbers.

Part 1 — What "maximum likelihood" means

1.1 Coin-flip intuition

Suppose you think a coin has unknown bias **p** (probability of heads). You flip it 10 times and see 7 heads, 3 tails.
Likelihood asks: *If the true bias were p, how probable is **this exact sequence**?*
For one flip: P(heads) = **p**, P(tails) = **1 − p**.
For 7 heads and 3 tails (order does not matter if flips are independent):

$$L(p) = p^7(1-p)^3$$

Try **p = 0.5**: $L = 0.5^{10} \approx 0.001$.
Try **p = 0.7**: $L = 0.7^7 \cdot 0.3^3 \approx 0.027$ — **higher**.
Maximum likelihood estimate $\hat{p}$ is the value that makes $L(p)$ as large as possible. Here $\hat{p} = 7/10 = 0.7$.

Same idea for draft: each player-season is a **biased coin** (drafted or not). The bias **p_i** depends on ability, congestion, λ, t. We search for λ and t that make the **whole panel** of yes/no outcomes as likely as possible.

1.2 Why we use log-likelihood

Multiplying thousands of small probabilities underflows on a computer. Take logs:

$$\ell = \log L = \sum_i \log P(Y_i \mid \cdots)$$

Maximize ℓ = same as maximize **L**. The script reports **ℓ** (negative numbers are normal).

1.3 What MLE does *not* guarantee

- It does **not** prove the model is **true** — only "best fit **under this story**."
- Wrong story (e.g. deterministic top-K only) → "optimal" parameters that are useless (flat likelihood).
- MLE is only as good as the **generative story** you write in Part 3.

Part 2 — Bernoulli trials (yes / no outcomes)

Each player-season has **one** outcome:

- **Y = 1** — drafted (ever, from college)
- **Y = 0** — not drafted

A **Bernoulli trial** is one yes/no draw with success probability **p**.

Outcome	Probability
Y = 1	**p**
Y = 0	**1 − p**

One compact formula (works for both):

$$P(Y \mid p) = p^Y (1-p)^{1-Y}$$

Check: Y = 1 → **p**; Y = 0 → **(1 – p)**.
Independence assumption: we multiply (or add logs) across players as if each row were its own coin flip. That ignores "exactly K NBA picks per year" correlation — PD21 accepts that tradeoff for v1 (Part 5, Part 9).

Part 3 — Building the model probability p_i

This is the chain from **inputs** (A, L^C) to **p** (draft probability). No unexplained jargon.

3.1 Inputs (fixed for a given γ)

For player *i* in season *s*:

Symbol	What it is
A_i	Ability (PPM z within season) — from box stats
L^C_i	Leave-one-out congestion on his roster — from viability map (depends on **γ**)
λ	How many "points" of appeal congestion costs (**fit**)
t	Temperature — scales how much ability differences matter (**fit**)

Roster membership is **fixed** (empirical NCAA). **ρ** is **not** inside this likelihood.

3.2 Step A — Draft appeal (a score *before* probabilities)

Define a **real number** for each player — think "how much the model likes him for draft **relative to peers**":

$$\text{appeal}_i = \frac{A_i}{t} - \lambda L_i^C$$

Why divide A by t?
If **t** is small, the same ability gap produces a **bigger** appeal gap (rankings are sharp). If **t** is large, ability differences get compressed (everyone looks similar).
Why subtract λ L^C?
More crowded roster → lower appeal. **λ** is the weight on that penalty.
*(Statisticians often call numbers like appeal_i "logits" before softmax. You can ignore that word: it is just **the score we exponentiate next**.)*

3.3 Step B — Softmax: scores → probabilities within a season

Probabilities must satisfy:

1. Each **p_i** between 0 and 1
2. **Sum to 1** over all players **in the same season**

Why exponentiate?
Appeal can be negative or positive. **exp(appeal)** is always positive, which we need for probabilities.
Why divide by a sum?
To force the total to 1.

$$p_i = \frac{\exp(\text{appeal}_i)}{\sum_{j \in \text{season } s} \exp(\text{appeal}_j)}$$

Substitute appeal:

$$p_i = \frac{\exp\left(A_i/t - \lambda L_i^C\right)}{\sum_{j \in s} \exp\left(A_j/t - \lambda L_j^C\right)}$$

Same thing, factored (helps intuition):

$$p_i = \frac{\exp(A_i/t)\,\exp(-\lambda L_i^C)}{\sum_j \exp(A_j/t)\,\exp(-\lambda L_j^C)}$$

Read it: high ability boosts the numerator; high congestion shrinks it; the denominator is the same "pool total" for everyone that season.
Critical implementation detail: compute this **separately for each season**. Do not softmax over all 38k rows at once. The code loops seasons.
Numerical stability: before exp(), subtract the max appeal in the season (log-sum-exp trick). Same **p_i**, safer on a computer.

3.4 What softmax does *not* do

- It does **not** set the top K players to p = 1/K and everyone else to 0.
- It does **not** force ∑ p_i = K (NBA slots). It forces ∑ p_i = **1**.
- So each **p_i** is a **small share** of a unit pie — typical player has p ≪ 1. MLE still works by pushing **drafted** players' shares up and **non-drafted** shares down.

Part 4 — Deriving the likelihood (full chain)

4.1 One player

Model says: player *i* is drafted with probability **p_i** (from Part 3).
Bernoulli (Part 2):

$$P(Y_i \mid p_i) = p_i^{Y_i}(1-p_i)^{1-Y_i}$$

- Drafted (Y_i = 1): factor **p_i** — model should have given him a decent chance.
- Not drafted (Y_i = 0): factor **(1 – p_i)** — model should not have given him too large a chance.

4.2 All players (independence)

$$L = \prod_i P(Y_i \mid p_i) = \prod_i p_i^{Y_i}(1-p_i)^{1-Y_i}$$

p_i depends on (λ, t, γ) through appeal and softmax.

4.3 Log-likelihood (what we maximize)

$$\ell(\lambda, t, \gamma) = \sum_i \Big[Y_i \log p_i + (1-Y_i)\log(1-p_i) \Big]$$

Drafted rows pull ℓ up when **p_i** increases (log p_i less negative).
Non-drafted rows pull ℓ up when **p_i** decreases (log(1 – p_i) less negative).
The optimizer searches **λ** and **t** (γ fixed or profiled) to climb ℓ.

4.4 Where K enters (and where it does not) — read carefully

Short answer: we do **not** run a second optimization that "plugs in K" to finalize λ̂ and t̂. The fit is **finished** when Bernoulli ℓ is maximized. K shows up in the **data**, in **sanity checks**, and in the **simulator** — not as a knob inside the likelihood formula.

See **Part 5** (two phases: fit → apply) and **Part 6** (tiny hand example).

In the likelihood itself:

- Real NBA drafts about **K** players per season from the pool.
- **Our formula does not contain K.** Softmax forces ∑ p_i = **1** per season, not K.

How K is still in the picture:

- Each season has **K_s** yeses in the data — that pattern is what ℓ rewards row by row.
- After the fit, we **check** top-**K_s** overlap (diagnostic — does not re-fit).
- In **sim**, we draw **exactly K** winners (Part 5C).

Part 5 — Two phases: fit the parameters, then use known K

Only **Phase A** changes λ and t.

Phase A — Fit λ̂ and t̂ (what the script optimizes)

Known going in:

- Each player has **Y = 1** (drafted) or **Y = 0** (not).
- Each season **s** has **K_s** = count of drafted players that year (count the Y = 1 rows).
- **A**, **L^C**, rosters — fixed inputs.

Unknown (what we estimate): **λ**, **t** (γ fixed at 18 in committed run).

Math (for each trial λ, t):

1. **appeal_i** = A_i/t − λ L^C_i
2. Softmax → **p_i** (sum to **1**, not K)
3. Bernoulli log-likelihood ℓ (Part 4.3)
4. Maximize ℓ over λ, t

Not a step: "Set ∑ p_i = K" or pass K into the optimizer.

Why it can work: There are **K_s** ones in the data. The fit pushes **p** up on drafted rows and down on others. Good λ, t → **highest p** players mostly match draftees (Alex: fit the success **distribution**, not "K sequential picks").

Output: λ̂, t̂, every **p_i**. **Parameters are final** — no Phase B re-estimation in v1.

Phase B — Use known K (after the fit)

B1. Top-K overlap (diagnostic — does not change λ̂, t̂)

Per season **s**:

1. **K_s** = number drafted (count Y = 1)
2. Sort players by fitted **p_i**
3. Take top **K_s** — model's implied draft class
4. Count overlap with actual draftees

Code: `topk_overlap()` in `pd21_draft_bernoulli_mle.py`. Mean overlap ≈ **12** picks/season on 2013–2021.

B2. Do not rescale p_i to sum to K (not in v1)

We do **not** multiply **p_i** by K_s. **p_i** stay softmax shares (sum to 1).

B3. Sim K-draw SELECT (same λ̂, t̂, different rule)

For generative draft pictures:

1. Compute **appeal** with fitted parameters
2. Weights **w_i** ∝ exp(appeal_i)
3. Draw **exactly K** players without replacement

K is **explicit** here. Same appeal formula as MLE; **different** selection mechanics than Bernoulli Phase A.

Step	K in formula?	Changes λ̂, t̂?	Purpose
A. Bernoulli MLE	No (K only via which Y=1)	**Yes**	Estimate λ, t
B1. Top-K check	K_s for overlap count	**No**	Sanity
B2. Rescale ∑p=K	—	—	**Not done**
B3. Sim K-draw	Yes — K winners	**No**	Generative sim

If K must be inside the fit: use **K-draw likelihood** (Part 10) — not v1 empirical.

Part 6 — Tiny worked example (one season, five players, K = 2)

Fix **λ = 1**, **t = 1** for the toy.

Player	A	L^C	Drafted? Y
1	2.0	0.5	**1**
2	1.0	0.5	**1**
3	0.0	0.5	0
4	−0.5	0.5	0
5	−1.5	0.5	0

K = 2 drafted (two Y = 1 rows). The fit never plugs "2" into the formula — this is just how many yeses appear in the data.

Step 1 — Appeal = A/t − λ L^C:

Player	Calculation	appeal
1	2.0 − 0.5	**1.5**
2	1.0 − 0.5	**0.5**
3	0.0 − 0.5	**−0.5**
4	−0.5 − 0.5	**−1.0**
5	−1.5 − 0.5	**−2.0**

Step 2 — Softmax: exponentiate, then divide by the season sum.

Player	exp(appeal) (≈)
1	4.48
2	1.65
3	0.61
4	0.37
5	0.14
Sum	**7.24**

Player	p_i = exp / sum (≈)
1	**0.619**
2	**0.228**
3	0.084
4	0.051
5	0.019

Check: 0.619 + 0.228 + 0.084 + 0.051 + 0.019 ≈ **1.00** (not K = 2).

Step 3 — Log-likelihood (players 1 and 2 drafted):

$$\ell = \log(p_1) + \log(p_2) + \log(1 - p_3) + \log(1 - p_4) + \log(1 - p_5)$$

$$\ell \approx \log(0.619) + \log(0.228) + \log(0.916) + \log(0.949) + \log(0.981) \approx -0.48 - 1.48 - 0.09 - 0.05 - 0.02 = \mathbf{-2.12}$$

Drafted players get the **high** p values → ℓ is reasonably high (not too negative). If the wrong players had been drafted — say only players 4 and 5 — you would be taking log of tiny p's on yes-rows and ℓ would collapse (≈ –6 or worse), pushing the optimizer toward different λ, t.
Step 4 — Phase B with K = 2: Sort by p: players **1, 2** on top. Actual draftees → **1, 2**. Overlap **2/2**, recall **100%** on this toy season.

Part 7 — Why not "top K only" (Rule C)?

Old hard rule: Y_i = 1 iff player *i* is in the top K by score.

Then for each player, P(Y_i) is **either 0 or 1** under the model that generated the data. Every observed outcome gets probability 1. The likelihood is **flat** in λ and t — no hill to climb.

That is why we moved to **probabilistic** SELECT (softmax + Bernoulli) before fitting.

Part 8 — How the fit is computed (algorithm)

Script: `sports/scripts/pd21_draft_bernoulli_mle.py`

8.1 Build panel

- POST-QC filters, min 20 minutes
- Columns: A, L^C (at chosen γ), Y_draft
- Group by season for softmax

8.2 Coarse grid (good starting point)

Try many (λ, t) pairs on a grid, e.g.:

- λ ∈ {0.5, 1.0, 1.5, 2.0, 2.5, 3.0}
- t ∈ {0.01, 0.1, 0.3, 1.0, 3.0, 10.0}

For each pair: compute all **p_i**, evaluate ℓ, keep the best.

8.3 Refine (L-BFGS-B)

Start from grid best. Numerical optimizer adjusts λ and t continuously to increase ℓ until improvement stops (11 iterations at γ = 18 in committed run).

Think: **smart hill-climbing** with bounds — not magic, not closed form.

8.4 γ profile (diagnostic)

Fix γ at each value in {5, 8, 10, 15, 18, 20, 30, 40, 60, 80, 100, 150, 200}:

1. Rebuild L^C (congestion depends on γ)
2. Re-run grid + refine for (λ, t)
3. Record best ℓ, $\hat{\lambda}$, $\hat{t}$

Plot ℓ vs γ → see flat region (γ not sharply identified). X-axis labels every grid γ on the profile PNG.

Part 9 — Honest caveats (know them, do not hide them)

9.1 Probabilities sum to 1, not K

Within a season, ∑ p_i = 1. If you treated Bernoullis literally, expected number of "successes" would be 1, not ~60–125 NBA picks.
What we still learn: who gets high **p** vs low **p** — the ranking and spread. PD21: match **highest-p** players to drafted set, not match draft counts.

9.2 Independence vs "exactly K winners"

Real draft: about K slots → negative correlation (more picks for others if one guy goes early, in a soft sense). Independent Bernoullis ignore that joint structure. K-draw likelihood (Part 10) would be tighter to the sim but harder to implement.

9.3 Small p_i for everyone

Typical p_i ≪ 1. MLE is still valid — it adjusts **relative** sizes. Interpret **p_i** as "share of season draft mass," not "literal NBA probability = p_i."

9.4 γ weakly identified

Profile shows ℓ flat for γ ≈ 18–100 while $\hat{\lambda}$, $\hat{t}$ drift slightly. Draft yes/no alone does not pin viability sharpness; we anchor γ = 18 for sim alignment.

Part 10 — Alternative: K-draw likelihood (if K were inside the fit)

Story: same appeal → weights w_i ∝ exp(appeal_i). Draw **exactly K** distinct players without replacement.
Likelihood is probability of the **observed drafted set**, not per-player Bernoulli with p_i = softmax share.
Use in **generative sim** / Pass B/C (Phase B3). **Empirical MLE v1** uses Bernoulli Phase A only. Alex expects small gap at MBB N.

Part 11 — PD20 temperature sweep vs MLE $\hat{t}$

	PD20 sweep	This MLE
Data	Simulated leagues	Real draft Y
Question	Does inverted-U survive soft SELECT?	What λ, t fit real outcomes?
t	Grid diagnostic	$\hat{t} \approx 1.07$ estimated

Do not quote PD20 grid t as the empirical $\hat{t}$.

Part 12 — Symbol cheat sheet (pipeline shorthand OK)

Symbol	Plain meaning
Y	1 = drafted, 0 = not
p	Model draft probability for that row
A	Ability (PPM z)
L^C	LOO congestion
λ	Congestion weight in appeal
t	Temperature (ability scaling)
γ	Viability sharpness (builds L^C)
ℓ	Log-likelihood (bigger = better fit)
appeal	A/t − λ L^C (score before softmax)

Avoid needing: "logit" = appeal; "softmax" = Part 3.3; "Bernoulli" = Part 2.

Part 13 — Files and commands

Item	Path
Overview	`MLE_fit_explainer.md`
This doc	`3-Master_Plan/MLE/MLE_basics.md`
Fit script	`sports/scripts/pd21_draft_bernoulli_mle.py`
JSON	`pd21_mle/PD21_draft_bernoulli_mle_2013_2021.json`
γ profile	`pd21_mle/PD21_draft_bernoulli_mle_2013_2021_gamma_profile.png`

Regenerate figure + slides:

```
python sports/scripts/build_pd21_mle_fit_slides.py --season-min 2013 --season-max 2021 --slides-only --force-figures
```

PDF when ready:

```
./scripts/convert_single_md_to_pdf.sh 3-Master_Plan/MLE/MLE_basics.md
./scripts/convert_single_md_to_pdf.sh 3-Master_Plan/MLE/MLE_fit_explainer.md
```

Edit notes (Charles)

- [x] Instructional rewrite — derive Bernoulli → appeal → softmax → ℓ
- [x] Explain why log, why exp, why season loop
- [x] Worked numeric toy example (Part 6 — five players, K = 2)
- [x] Two-phase K story (Part 5 — fit vs apply)
- [ ] Joint (λ, γ, t) MLE pass when ready